পর্ব ৮ · ক্লাস ৩৩

দস্তাবেজ — কে পড়বে, কী কাজে

ব্যবহারবিধি · নতুনদের পথ · প্রমাণের দস্তাবেজ · স্ক্রিনশট · বাংলায় লেখা

IMS · ক্লাস ৩৩ · স্লাইড ১–১৫

শুরু

আপনার README এখনো ভুল বলছে

আসল উদাহরণ, কল্পনা নয়

```
# README.md — আজ যা লেখা আছে

## What exists right now — Slice 1

| Students, academics, attendance,
| lessons, assessments, reports, audit
|                    | **not built yet** |

# বাস্তবে: ১১টি অ্যাপ, ১০৩ টেস্ট,
# ৫৫ পাতা, ৭৭ স্ক্রিনশট — সব চলছে
```

এটাই ডকুমেন্ট-ড্রিফট

কোড এগিয়েছে, দস্তাবেজ দাঁড়িয়ে আছে। মিথ্যা বলা নয় — **ভুলে যাওয়া**।

কেন ভয়ঙ্কর

নতুন ডেভেলপার প্রথমেই README পড়ে — ভুল তথ্য পেলে হয় অপ্রয়োজনীয় কাজ করে, নয়তো ভরসা হারায়।

প্রতিকার

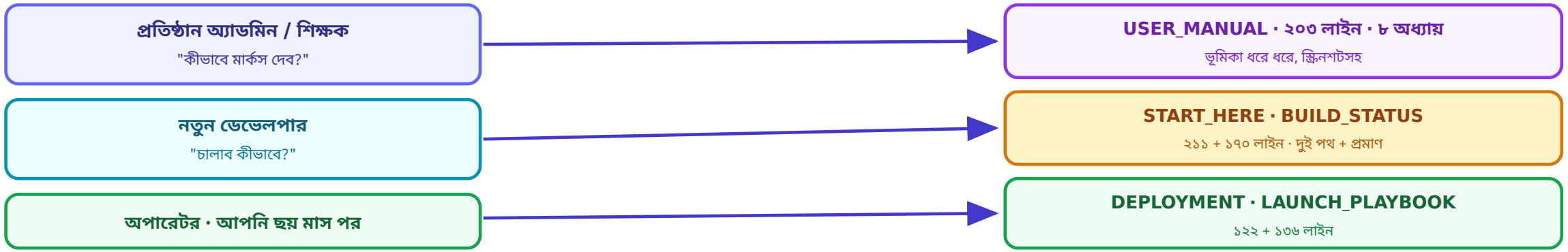
সংখ্যা হাতে না লিখে **স্ক্রিপ্ট** তৈরি করা — এই ক্লাসের শেষ অংশে সেটাই দেখব।

শিক্ষকের নোট: ক্লাস শুরুর এই আঘাতটাই দরকারি: "আপনার নিজের প্রজেক্টেও এমন লাইন আছে।" README-র এই অংশটি প্রজেক্টে সত্যিই আছে — শিক্ষার্থীদের দেখিয়ে দিন।

মানচিত্র

চার পাঠক, চার রকম দস্তাবেজ

একটি দস্তাবেজ সবাইয়ের জন্য নয়



একটি দস্তাবেজে সব লেখার চেষ্টা করলে প্রত্যেকে অপ্রয়োজনীয় অংশ পড়ে থেমে যায় — আর কেউ পুরোটা পড়ে না।

শিক্ষকের নোট: জিজ্ঞেস করুন: "আপনার প্রজেক্টের দস্তাবেজ কে পড়ে?" উত্তর না মিললে দস্তাবেজটি সাজসজ্জা, প্রয়োজন নয়।

তালিকা

ছয়টি ফাইল, ৯০০ লাইন

প্রতিটির একটি নির্দিষ্ট কাজ

ফাইল	লাইন	পাঠক	প্রশ্ন যার উত্তর দেয়
`README.md`	৫৮	সবাই	এটি কী, কী দিয়ে বানানো — **এখন বাসি**
`START_HERE.md`	২১১	নতুন ডেভেলপার	নিজের মেশিনে চালাব কীভাবে
`USER_MANUAL.md`	২০৩	প্রতিষ্ঠান / শিক্ষক	কোন পর্দায় কী করব
`BUILD_STATUS.md`	১৭০	আপনি ও গ্রাহক	কী তৈরি, কী বাকি, কোন প্রমাণে
`LAUNCH_PLAYBOOK.md`	১৩৬	আপনি (অনবোর্ডিং)	স্কুল আনব কীভাবে, টাকা কীভাবে নেব
`DEPLOYMENT.md`	১২২	অপারেটর	সার্ভারে তুলব কীভাবে

ছয়টি ফাইলই মূল (Markdown) — একই লেখা HTML/PDF-এও যায় (`docs/manual/user-manual-bn.html` , `docs/IMS_User_Manual_BN.pdf`)। এক উৎস, অনেক রূপ।

শিক্ষকের নোট: টেবিলের শেষ কলামটাই মূল — প্রতিটি ফাইলের **একটি প্রশ্ন** থাকা দরকার। প্রশ্ন না থাকলে ফাইলটি বাড়ে, কাজে লাগে না।

ব্যবহারবিধি

ভূমিকা ধরে সাজানো ম্যানুয়াল

আটটি অংশ — শুরুতে "কে কী পাবে"

অংশ	বিষয়
১–২	শুরু করার আগে · লগইন, পাসওয়ার্ড, ওয়ার্কস্পেস
অধ্যায় ১	প্রতিষ্ঠান অ্যাডমিন
অধ্যায় ২	কোঅর্ডিনেটর (স্কোপ)
অধ্যায় ৩	শিক্ষক
অধ্যায় ৪	শিক্ষার্থী
অধ্যায় ৫	প্লাটফর্ম অ্যাডমিন
অধ্যায় ৬	সাবস্ক্রিপশন, ট্রায়াল, পেমেন্ট
৭–৮	সমস্যা ও সমাধান · পরিশিষ্ট

কেন ভূমিকা দিয়ে

শিক্ষক "প্রতিষ্ঠান তৈরি" অংশ পড়তে চান না। ভূমিকা-ভিত্তিক ভাগ = কম পড়া, দ্রুত উত্তর।

শেষে সমস্যা-সমাধান

"লগইন হচ্ছে না", "মার্কস সংরক্ষণ হচ্ছে না" — বাস্তব প্রশ্ন দিয়ে শুরু, ব্যাখ্যা পরে।

ফাঁক

ম্যানুয়ালের সঙ্গে স্ক্রিনশটের মিল হাতে রাখা হয় — UI বদলালে পুরোনো ছবি থেকে যায়।

শিক্ষকের নোট: ক্লাসে ম্যানুয়ালের একটি অধ্যায় খুলে দেখান — তারপর জিজ্ঞেস করুন: "এটি পড়ে একজন শিক্ষক সত্যিই কাজ করতে পারবেন?"

নিয়ম

ব্যবহারবিধি লেখার পাঁচ নিয়ম

লেখা ভালো না হলে ব্যবহারকারী ফোন করবে — সেটাই আসল খরচ

নিয়ম	খারাপ	ভালো
এক ধাপে একটি কাজ	"মার্কস দিন এবং প্রকাশ করুন এবং রিপোর্ট দেখুন"	তিনটি আলাদা ধাপ, তিনটি ছবি
পর্দার নাম হুবহু	"সেটিংসে যান"	"Classes & subjects → New class"
ফলাফল আগে বলুন	"ফর্ম পূরণ করুন"	"ফর্ম পূরণ করলে ক্লাস তালিকায় দেখা যাবে"
ভুল হলে কী হবে	কিছুই না	"অনুপস্থিত ছাত্রে নম্বর দিলে সংরক্ষণ হবে না"
স্ক্রিনশট সত্যি হতে হবে	পুরোনো ছবি	চলমান অ্যাপ থেকে তোলা (`shoot.py`)

পরীক্ষা: কাউকে শুধু ম্যানুয়াল দিয়ে কাজটি করতে দিন — আটকে গেলে সেই জায়গাটাই আবার লিখতে হবে। (এটাই bus-factor পরীক্ষা, ক্লাস ৩২-এর।)

শিক্ষকের নোট: শিক্ষার্থীদের জোড়ায় ভাগ করে একটি অধ্যায় লিখিয়ে অন্য জনকে দিয়ে চালিয়ে দেখান — সবচেয়ে দ্রুত শেখার উপায়।

স্ক্রিনশট

৭৭টি ছবি, একটি স্ক্রিপ্ট

হাতে তোলা ছবি পুরোনো হয় — স্ক্রিপ্টের ছবি নয়

```
# docs/shoot.py — চলমান অ্যাপ থেকে ছবি
BASE = "http://127.0.0.1:8000"
DESKTOP = {"width": 1440, "height": 980}
MOBILE  = {"width": 390,  "height": 844}

def login(page, email):        # আসল লগইন
    page.fill("#id_username", email)
    page.fill("#id_password", PASSWORD)
    page.click("button[type=submit]")

def shot(page, name):          # নেটওয়ার্ক শান্ত হলে ছবি
    page.wait_for_load_state("networkidle")
    page.screenshot(path=f"{OUT}/{name}")
```

ফোল্ডার	সংখ্যা
`screenshots/` (মূল)	৫৩
`screenshots/saas/`	১৩
`screenshots/roles/`	৯
`screenshots/manual/`	৫
মোট	**৭৭ PNG**

নামকরণ: `30_attendance_take.png` — ক্রম + পর্দার নাম; খোঁজা সহজ, অধ্যায়ের ক্রমও মেলে।

শিক্ষকের নোট: `shoot.py` চালাতে সার্ভার ও ডেমো ডেটা দরকার — ক্লাসের আগে `bash run.sh --reset` করে রাখুন।

প্রমাণ

পরিকল্পনা বনাম প্রমাণ

৫৭ ফিচার · ৫৪টির ছবি-প্রমাণ · দুই আলাদা ফাইল

```
# docs/build_status.py — প্রমাণের মানচিত্র
BUILT = {
    "F-01": "01_login.png",
    "F-02": "02_register.png",
    "F-05": "07_profile.png",
    "F-08": "05_switch_workspace.png",
    "F-09": "08_create_institution.png",
    "F-13": "10_org_admin_dashboard.png",
    "F-16": "04_teacher_dashboard.png",
    ...
}

# ডকুমেন্টেশনে লেখা আছে:
# "Kept separate from catalogue_data.py on purpose:
#  the catalogue is the plan, this file is the evidence."
```

দুই ফাইল, দুই প্রশ্ন

`catalogue_data.py` (৩১ KB) — কী হওয়ার কথা (৫৭ ফিচার)।

`build_status.py` — কোনটির ছবি-প্রমাণ আছে (৫৪টি)।

কেন আলাদা

"ছবি তালিকা আবার তৈরি করলে ফিচার-স্পেসিফিকেশন নষ্ট হবে না" — ডিজাইনের সিদ্ধান্ত, কোডেই লেখা।

ফল

দাবি করা আর প্রমাণ করা **একই জায়গায়** থাকে — তাই ফাঁক লুকায় না, গোনা যায়।

শিক্ষকের নোট: এটি এই কোর্সের মূল দর্শন: "দাবি + প্রমাণ"। শিক্ষার্থীকে বলুন নিজের প্রজেক্টে F-নম্বরের বদলে নিজের ফিচার-নাম দিয়ে এমন একটি মানচিত্র বানাতে।

দস্তাবেজও কোড

পাঁচটি জেনারেটর স্ক্রিপ্ট

হাতে লেখা সংখ্যা বাসি হয় — চালানো সংখ্যা নয়

স্ক্রিপ্ট	আকার	কী তৈরি করে
`docs/catalogue_data.py`	৩১ KB	ফিচার-ক্যাটালগের উপাদান (৫৭টি)
`docs/gen_feature_catalogue.py`	৩৪ KB	ক্যাটালগ PDF (স্ক্রিনশটসহ)
`docs/gen_proof_pdf.py`	১৯ KB	`Build_Proof.pdf` — প্রমাণের নথি
`docs/gen_roadmap.py`	২৮ KB	ক্লাস-রোডম্যাপ নথি
`docs/build_status.py`	৩.৩ KB	ফিচার → স্ক্রিনশট মানচিত্র
`docs/shoot.py`	৬.২ KB	৭৭টি স্ক্রিনশট (ডেস্কটপ + মোবাইল)

সঠিক ক্রম: `run.sh` → `shoot.py` (ছবি) → `gen_*` (নথি) — ছবি ছাড়া নথি তৈরি হলে ফাঁকা পাতা।

ফাঁক: এই ছয়টি চালানোর এক-কমান্ড স্ক্রিপ্ট (`rebuild_docs.sh`) এখনো নেই — কেউ একটা ভুলে যায়।

শিক্ষকের নোট: ক্লাসে `python docs/build_status.py` চালিয়ে দেখান — সংখ্যা নিজে থেকে হালনাগাদ হচ্ছে কি না। এই একটি মুহূর্ত অনেকের ধারণা বদলায়।

নতুনদের পথ

`START_HERE` — দুই পথ, এক গন্তব্য

২১১ লাইন, যাতে প্রথম দিনে কেউ আটকে না থাকে

পথ A — SQLite (প্রস্তাবিত প্রথমবার)

শুধু Python লাগে · `bash run.sh` · একটি ফাইল ডেটাবেস · ৩ মিনিটে চলা শুরু

পথ B — PostgreSQL (আসল ভঙ্গি)

ডেটাবেস সার্ভার + `.env` · প্রোডাকশনের মতো · ডেভেলপার যেটা শিখবে সেটাই সার্ভারে

যা দুই পথেই আছে

ভার্চুয়াল এনভায়রনমেন্ট · `pip install -r requirements.txt` · `seed_demo` · চারটি লগইন

```
# পথ A — তিন কমান্ড
bash run.sh
#  http://127.0.0.1:8000/

# পথ B
python3 -m venv .venv
. .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env      # মান বসান
python manage.py migrate
python manage.py seed_demo
python manage.py runserver
```

শিক্ষকের নোট: দুই পথ রাখার কারণ: প্রথমবার যেটা সহজ, সেটাই শেখার পথ; পরে আসল ভঙ্গি। একটাই পথ রাখলে নতুনদের অর্ধেক আটকে যাবে।

সংখ্যার খাতা

`BUILD_STATUS` — সংখ্যা ও প্রমাণ

প্রতিটি সংখ্যার পাশে কমান্ড, প্রতিটি দাবির পাশে টেস্ট

```
## Verified numbers
python manage.py test     ->  90 tests, 90 OK
python smoke_test.py      ->  55 pages x 4 roles
manage.py check --deploy  ->  no issues

## Remaining work (not faked, not skipped)
1. Invitation workflow
2. Platform console
4. Backup & restore console
5. Security attack suite
```

এটি বাসি হয়ে গেছে

এখন ৯০ নয়, **১০৩ টেস্ট** — সংখ্যাটি লেখার সময়ের সত্যি, আজকের নয়।

হালনাগাদের নিয়ম

প্রতিটি রিলিজে তিনটি সংখ্যা আবার চালান: `test` · `smoke_test` · `check --deploy`।

দর্শন

"not faked, not skipped" — যা হয়নি, তাও লেখা। এটাই দস্তাবেজকে বিশ্বাসযোগ্য করে।

শিক্ষকের নোট: এই স্লাইডটি ভালো ও খারাপ দুই দিকই দেখায়: প্রমাণ-খাতার ধারণা সঠিক, কিন্তু হালনাগাদ না হলে সেটিও মিথ্যা বলতে শুরু করে।

সততা

সৎ ভাষায় লেখা

যা নেই তা লেখাই দস্তাবেজের সবচেয়ে দামি অংশ

প্রজেক্টে যা লেখা আছে	মানে
"Remaining work (**not faked, not skipped**)"	ফাঁক লুকানো হয়নি, বাদও দেওয়া হয়নি
"It has **not** yet been deployed to a public host"	রিহার্সাল হয়েছে, ডিপ্লয় হয়নি — পার্থক্য স্পষ্ট
"**Demo data removed** — DemoPass!2026 gone"	চেকলিস্টে দায়িত্ব, অনুমান নয়
"A backup you have never restored is not a backup"	নীতি এক বাক্যে, মনে রাখার মতো

যা লিখবেন না: "সব কাজ করছে" · "শীঘ্রই আসছে" · যাচাই না করা সংখ্যা

যা লিখবেন: "কাজ করছে (টেস্ট ১০৩, আজ ২০২৬-১১-০৫)" · "আসছে: কবে, কীভাবে বোঝা যাবে"

শিক্ষকের নোট: সৎ দস্তাবেজের সবচেয়ে বড় লাভ: গ্রাহক একবার ভুল তথ্য পেলে আর কিছুতেই ভরসা করে না। "না" বলা বিশ্বাসযোগ্যতা বাড়ায়।

ভাষা

বাংলায় টেকনিক্যাল লেখা

শব্দ বাংলা, পরিভাষা ইংরেজি — মিশ্রণ নিয়ম মেনে

যা অনুবাদ করবেন না	কারণ
`migrate` · `commit` · `deploy`	টার্মিনালে যেটা লেখে, দস্তাবেজেও সেটাই
`middleware` · `queryset`	বাংলা করলে খুঁজে পাওয়া যায় না
পর্দার নাম (ইংরেজি)	ব্যবহারকারী পর্দায় ইংরেজিই দেখেন

যা বাংলায় লিখবেন

ব্যাখ্যা, কারণ, সতর্কতা, ধাপের ক্রিয়া — "সংরক্ষণ করুন", "যাচাই করুন", "ফেরান"। এগুলো পড়ার গতি বাড়ায়।

```
# ভালো মিশ্রণ
"মার্কস সংরক্ষণ করতে Save চাপুন।
অনুপস্থিত ছাত্রে নম্বর দিলে ফর্ম
গ্রহণ করবে না।"

# খারাপ
"মার্কসংরক্ষণকার্যসম্পাদনসূচক বোতাম"

# আরও খারাপ
"Click the save button to persist
the marks record in the database"
```

শিক্ষকের নোট: শিক্ষার্থীরা দুই চরমে যায় — পুরো ইংরেজি, বা কঠিন বাংলা। মাঝের পথটাই ব্যবহারিক; একটি প্যারাগ্রাফ একসাথে লিখে দেখান।

অনুশীলন ও আলোচনা

২০ মিনিট — একটি পাতা লিখুন

চারটি কাজ, সবই ব্যবহারযোগ্য

১. একটি কাজের পাতা

"উপস্থিতি নেওয়া" — ৫ ধাপ, পর্দার নাম হুবহু, স্ক্রিনশটের জায়গা চিহ্নিত।

২. প্রশ্ন-উত্তর পাঁচটি

"লগইন হচ্ছে না" ধরনের বাস্তব সমস্যা, সংক্ষিপ্ত সমাধান।

৩. সত্যি সংখ্যা

নিজের প্রজেক্টে `test` চালিয়ে সংখ্যা লিখুন — সাথে তারিখ।

৪. প্রমাণ-মানচিত্র

পাঁচটি ফিচার → কোন স্ক্রিনশট প্রমাণ করে (ছোট ডিকশনারি)।

- দস্তাবেজ কে পড়বে?
- সংখ্যা হাতে লিখলে কী হয়?
- কেন পরিকল্পনা ও প্রমাণ আলাদা ফাইল?
- টেকনিক্যাল শব্দ অনুবাদ করলে?
- যা নেই তা লেখা কেন দরকার?

শিক্ষকের নোট: চারটি কাজের ফল জমা নিন — পরের ক্লাসে সেগুলোই আরও পরিষ্কার করা হবে (পর্ব ৮-এর দ্বিতীয় ক্লাস)।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

দস্তাবেজের প্রথম ক্লাসের সারাংশ

নীতি	প্রমাণ
প্রতি পাঠকের জন্য আলাদা দস্তাবেজ	২১১ · ২০৩ · ১৭০ · ১৩৬ · ১২২ · ৫৮ লাইন
এক ধাপে একটি কাজ, পর্দার নাম হুবহু	USER_MANUAL-এর ৮ অধ্যায়
ছবি স্ক্রিপ্টে, হাতে নয়	`shoot.py` · ৭৭ PNG
পরিকল্পনা ও প্রমাণ আলাদা রাখুন	৫৭ ফিচার · ৫৪ ছবি-প্রমাণ
সংখ্যা জেনারেট করুন, লিখবেন না	`docs/gen_*.py` পাঁচটি
যা নেই তা লিখুন	"not faked, not skipped"

পরের ক্লাসে: ডেভেলপার-দস্তাবেজ — কোডের ভেতরের ব্যাখ্যা, চেঞ্জলগ, এবং "কেন এভাবে" লেখার কৌশল।

শিক্ষকের নোট: শেষে একটি কাজ দিন: "আপনার README-এর বাসি লাইনটি আজই ঠিক করুন" — ছোট, কিন্তু বদল দেখা যায়।

পর্ব ৮ · ক্লাস ৩৪

কোডের ভেতরের দস্তাবেজ

docstring · "কেন" মন্তব্য · URL মানচিত্র · CHANGELOG · ADR · CONTRIBUTING

IMS · ক্লাস ৩৪ · স্লাইড ১৬–৩০

শুরু

কোডে দস্তাবেজ কতটা আছে?

আজ গোনা সংখ্যা — ৮৭ ফাইল, ৫৭৬ def/class

৭৫টি docstring — ১৩.০%

৫৭৬টি ফাংশন/ক্লাসের মধ্যে মাত্র ৭৫টির ব্যাখ্যা আছে। বাকিগুলো নামের ওপরই ভরসা করছে।

মডিউল-স্তরে ভালো

প্রায় প্রতিটি অ্যাপের `models.py` / `views.py` -এর শুরুতে উদ্দেশ্য লেখা — যেমন "Exams, marks, grading and results (C112–C119)".

ফাংশন-স্তরে দুর্বল

জটিল helper-গুলোর ব্যাখ্যা নেই — পঠনযোগ্যতা নামের ওপর নির্ভরশীল। এই ফাঁকটাই আজকের কাজ।

সংখ্যা বাড়ানোই লক্ষ্য নয় — **যেখানে নাম বলতে পারে না, সেখানেই** ব্যাখ্যা দরকার। সব ফাংশনে docstring দিলে তা আবার পড়া হয় না।

শিক্ষকের নোট: পরিমাপটি দেখান: `ast` দিয়ে ২০ লাইনের স্ক্রিপ্টেই এই সংখ্যা বের হয় — শিক্ষার্থীরা নিজের প্রজেক্টে চালাতে পারবে।

মডিউল-ডকস্ট্রিং

ফাইলের শুরুতে তিনটি প্রশ্নের উত্তর

এটি কী · নিয়ম কী · কেন এখানে

```
"""One place that answers: 'what may this person
see?' (production hardening, C126).

Every list view asks these helpers instead of
writing its own filter. That is how a role bug is
fixed once instead of in nine views.

    admin       -> everything in the workspace
    coordinator -> only the departments / classes
                   assigned to them
    teacher     -> only classes assigned to teach
    student     -> only their own record
"""
# apps/core/scoping.py
```

প্রথম লাইনে উদ্দেশ্য

"এক জায়গা যেখানে উত্তর আছে: এই মানুষটি কী দেখতে পারে?"

তারপর নিয়ম

চার ভূমিকার চারটি স্কোপ — টেবিলের মতো, চোখে পড়ে।

শেষে "কেন"

"নয়টি ভিউয়ে আলাদা করে লিখলে বাগ নয় জায়গায় ঠিক করতে হতো।"

যা লিখবেন না

"এই ফাইলে স্কোপিং ফাংশন আছে" — সেটা ফাইলের নামেই আছে।

শিক্ষকের নোট: এই docstring-টি প্রজেক্টে সত্যিই আছে — পড়ে দেখান। তারপর জিজ্ঞেস করুন: "এটি না থাকলে নতুন ডেভেলপার কত সময় নষ্ট করত?"

"কেন" মন্তব্য

যেখানে কোড নিজেই বিভ্রান্তিকর

অদ্ভুত দেখায় এমন লাইনের পাশে কারণ লিখুন

```
# apps/billing/middleware.py
# The membership that matters is for the workspace in
# the URL -- the session one can belong to a different
# institution entirely (admin of one, coordinator of
# another), which is exactly how this branch was wrong
# the first time.

# apps/audit/models.py
def log(...):
    """Write one audit row. Never raises: logging must
    not break a workflow."""
```

মূল্য

এই মন্তব্য না থাকলে পরের কেউ "অপ্রয়োজনীয়" ভেবে সরিয়ে দিত — আর বাগটি ফিরে আসত।

সূত্র

"this is exactly how this branch was wrong the first time" — সত্যি ঘটে যাওয়া বাগের স্মৃতি।

নিয়ম

মন্তব্যে "কী" নয়, "কেন" — এবং সম্ভব হলে ঘটে যাওয়া সমস্যার কথা।

শিক্ষকের নোট: বলুন: "সবচেয়ে দামি মন্তব্য সেই যেটি ভবিষ্যতের কাউকে একই বাগ করতে বাধা দেয়।" নিজের প্রজেক্টে এমন একটি খুঁজতে বলুন।

ডকস্ট্রিং কখন

কোথায় ব্যাখ্যা দরকার, কোথায় নয়

নামই যদি বলে দেয়, docstring বাড়তি

দরকার নেই

`def can_manage_workspace(membership):`
নামই বলছে — প্রতিষ্ঠান অ্যাডমিন কি না।

`def visible_classes(user, workspace):`
স্কোপিং ফাইলে আছে, উদ্দেশ্য স্পষ্ট।

দরকার আছে

`def is_writable(workspace):` — কোন পাঁচ অবস্থায় সত্য, তা দ্রুত দেখা দরকার।

`def _next_number(prefix):` — ফরম্যাট, প্রতিযোগিতা ও ব্যর্থতার নিয়ম ব্যাখ্যা দরকার।

যেখানে **নিয়ম** আছে (নিরাপত্তা, টাকা, তারিখ) — সেখানে ব্যাখ্যা।

নিয়ম: **যে ফাংশনে ভুল হলে টাকা বা নিরাপত্তা ক্ষতিগ্রস্ত হয়** — সেখানে সবসময় ব্যাখ্যা। বাকিতে নাম ও টেস্টই যথেষ্ট।

শিক্ষকের নোট: এটি সময় বাঁচানোর নিয়ম: সব ফাংশনে docstring লিখতে গেলে কেউ কিছুই লিখবে না। অগ্রাধিকার দিন গুরুত্বপূর্ণ জায়গায়।

মানচিত্র

URL মানচিত্র — ৮৫টি রুট, ১১ অ্যাপ

নতুন ডেভেলপারের প্রথম প্রশ্ন: কোন পর্দা কোথায়?

অ্যাপ	রুট	প্রধান পর্দা
`billing`	১৮	প্ল্যান · পাবলিক প্রাইসিং · সাবস্ক্রিপশন · ইনভয়েস · প্লাটফর্ম কনসোল
`academics`	১৩	শিক্ষাবর্ষ · টার্ম · বিভাগ · ক্লাস · বিষয় · অ্যাসাইনমেন্ট
`lessons`	১৩	সিলেবাস · টপিক · হোমওয়ার্ক · জমা
`accounts`	৯	লগইন · রেজিস্ট্রেশন · প্রোফাইল · পাসওয়ার্ড
`assessments`	৯	পরীক্ষা · নম্বর এন্ট্রি · প্রকাশ · মার্কশিট
`reports` · `students`	৭ + ৭	রিপোর্ট · CSV/XLSX · ছাত্র তালিকা ও প্রোফাইল
`attendance` · `audit` · `workspaces` · `core`	৪ · ২ · ২ · ১	উপস্থিতি · অডিট · সুইচ/তৈরি · ড্যাশবোর্ড

এই ছকটি কোথাও লেখা নেই — `grep 'path('` চালিয়ে তৈরি হয়েছে। `docs/url-map.md` বানানো পরের কাজ; নতুনদের জন্য এটি সবচেয়ে দ্রুত মানচিত্র।

শিক্ষকের নোট: ক্লাসেই ছকটি তৈরি করে দেখান: `for f in apps/*/urls.py; do echo $(dirname $f) $(grep -c 'path(' $f); done`

নামকরণ

নামই যখন দস্তাবেজ

ভালো নাম লাইন কমায়, খারাপ নাম মন্তব্য বাড়ায়

IMS-এর নাম	কী বলে
`test_student_only_gets_published_homework`	দাবিটাই নামে লেখা — ব্যর্থ হলে বোঝা যায়
`test_server_refuses_the_third_student`	সিট সীমার নিয়ম, ঠিক কোথায় ভাঙে
`visible_homework` · `is_writable`	ক্রিয়া + উদ্দেশ্য; বাড়তি ব্যাখ্যা লাগে না
`views_platform.py`	ফাইলটি প্লাটফর্ম-কনসোলের — ভাগ করা সহজ
`seed_demo` · `billing_report` · `sync_domains`	ম্যানেজমেন্ট কমান্ডের কাজ নামেই

পরীক্ষা: নাম শুনে কাজ বোঝা যায়? না হলে নাম বদলান — মন্তব্য লেখার আগে।

দুই অ্যাপে একই টেস্ট-নাম:
`test_teacher_without_assignment_sees_no_students` — রিপোর্টে গুলিয়ে যায় (ক্লাস ২৬-এ চিহ্নিত)।

শিক্ষকের নোট: শিক্ষার্থীদের সবচেয়ে খারাপ তিনটি নাম লিখতে বলুন, তারপর নতুন নাম প্রস্তাব করতে — দ্রুত উন্নতি দেখা যায়।

CHANGELOG

রিলিজ নোট — এখনো নেই, তবে দরকার

গ্রাহকের প্রথম প্রশ্ন: "নতুন কী এল?"

```
# CHANGELOG.md (প্রস্তাবিত)

## 1.4.0 — 2026-12-01
### যোগ হয়েছে
+ invitation workflow (F-10..12)
+ CSV student import

### বদলেছে
* academics tests 52 -> 90

### মাইগ্রেশন
! students 0004_add_invitation

### ঠিক হয়েছে
# marks entry rejected 0 as absent

## 1.3.2 — 2026-11-20
### ঠিক হয়েছে
# CSS cache busting on redeploy
```

আজকের অবস্থা

১৯টি কমিট আছে, `CHANGELOG.md` বা git tag নেই — "কোন সংস্করণে কী ছিল" প্রশ্নের উত্তর নেই।

কেন মাইগ্রেশন আলাদা লাইন

রিলিজ নোট পড়ে অপারেটর ব্যাকআপ নেয় — মাইগ্রেশন উল্লেখ না থাকলে নেয় না।

কে লেখে

যে রিলিজ করে — কারণ সে-ই জানে কী বদলেছে। পরে লিখতে গেলে ভুলে যাবে।

শিক্ষকের নোট: বলুন: "চেঞ্জলগ গ্রাহকের জন্য নয় শুধু, নিজের জন্যও — ছয় মাস পর নিজেই ভুলে যাবেন।" প্রথম এন্ট্রিটি ক্লাসেই লিখিয়ে নিন।

ADR

`docs/adr/` — সিদ্ধান্তের স্থায়ী ঠিকানা

এক সিদ্ধান্ত, এক ফাইল, কখনো সম্পাদনা নয়

```
docs/adr/
  0001-modular-monolith.md
  0002-custom-user-email-login.md
  0003-billing-inside-the-app.md
  0004-subscription-gate-middleware.md
  0005-custom-domain-verified-only.md

# প্রতিটির গঠন
## প্রসঙ্গ
## সিদ্ধান্ত
## বাদ পড়া বিকল্প
## পরিণতি
## তারিখ ও অবস্থা (গৃহীত / বাতিল)
```

৫টি আসল সিদ্ধান্ত, ৫টি ফাইল

প্রতিটির পক্ষে-বিপক্ষে যুক্তি কোডে ছড়িয়ে আছে — এক জায়গায় আনলেই নতুনদের সময় বাঁচে।

সম্পাদনা করবেন না

সিদ্ধান্ত বদলালে নতুন ADR লিখুন, পুরোনোটির অবস্থা "বাতিল" করুন — ইতিহাস রাখুন।

সংখ্যাই ক্রম

`0004` থেকে বোঝা যায় কোনটি আগে — তারিখ মুখস্থ করতে হয় না।

শিক্ষকের নোট: ক্লাসে `docs/adr/0001-modular-monolith.md` তৈরি করে দেখান — ২০ মিনিটের কাজ, কিন্তু বছরের জন্য।

CONTRIBUTING

দলে যোগ দেওয়ার এক পাতা

নিয়ম লিখিত না হলে নিয়ম নেই

```
# CONTRIBUTING.md

## প্রথম দিন
1. START_HERE.md পড়ে অ্যাপ চালান
2. manage.py test -> ১০৩ সবুজ
3. BUILD_STATUS.md পড়ুন

## কাজ শুরু
* শাখা: fix/marks-boundary
* ছোট কমিট, বার্তায় "কী + কেন"
* টেস্ট ছাড়া PR নয়

## মাইগ্রেশন থাকলে
* রিলিজ নোটে উল্লেখ করুন
* রিভিউয়ারকে আলাদা করে বলুন

## মার্জের শর্ত
টেস্ট সবুজ + একজন রিভিউ + দস্তাবেজ হালনাগাদ
```

কেন দরকার

"কেউ বলে দেবে" ধরে নিলে নতুন সদস্য ভুল করে, আর আপনি বিরক্ত হন — দুটোই এড়ানো যায়।

এক পাতা যথেষ্ট

৫০ লাইনের বেশি লিখলে কেউ পড়ে না — আজকের তিনটি নিয়মই যথেষ্ট।

আজ নেই

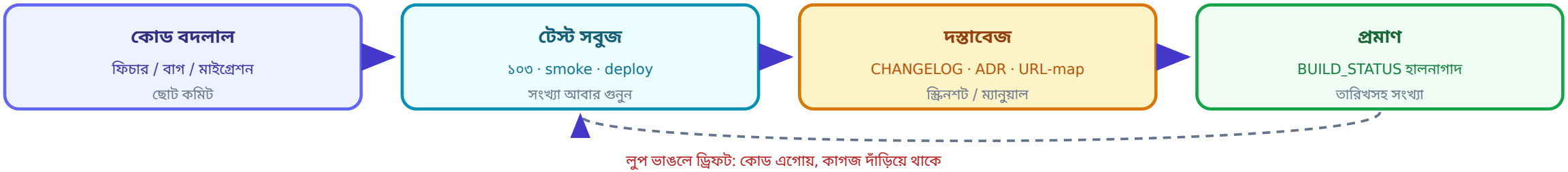
প্রজেক্টে `CONTRIBUTING.md` বা `CHANGELOG.md` নেই — দুটিই প্রথম সপ্তাহের কাজ।

শিক্ষকের নোট: শিক্ষার্থীদের নিজের দলে "মার্জের শর্ত" তিনটি লিখতে বলুন — এটাই সবচেয়ে কাজে লাগা অংশ।

ড্রিফট

কোড আর দস্তাবেজ মিলিয়ে রাখার লুপ

প্রতি রিলিজে একই চার ধাপ



যাচাইয়ের কমান্ড: `makemigrations --check --dry-run` · `test` · `check --deploy` · `build_status.py`

ফাঁক: দস্তাবেজ হালনাগাদ এখনো হাতে — CI বা প্রি-কমিট হুক থাকলে ভুলে যাওয়া বন্ধ হতো।

শিক্ষকের নোট: এই চার ধাপ রিলিজ-চেকলিস্টে বসিয়ে দিন — কাগজে থাকলে মনে থাকবে, মুখে থাকলে থাকবে না।

কমান্ডের খাতা

নয়টি স্ক্রিপ্ট, একটি ছক

যে কেউ যেন চালাতে পারে

কমান্ড	কী করে	কখন চালাবেন
`bash run.sh`	SQLite ডেমো চালু (৮০০০)	প্রথমবার / ক্লাসে
`bash run.sh --reset`	ডেটাবেস নতুন করে বানায়	ডেমো ডেটা নষ্ট হলে
`bash build.sh`	install + collectstatic + migrate	হোস্টে রিলিজ
`bash docs/deploy_check.sh`	Gunicorn :8010 + ৫টি চেক	ডিপ্লয়ের আগে
`ops/backup.sh` · `ops/restore.sh`	ডাম্প · scratch-এ রিস্টোর	রাত ২টা · মাসে একবার
`manage.py seed_demo` · `seed_plans`	ডেমো ডেটা · প্ল্যান	কখনো প্রোডাকশনে নয় (ডেমো)
`manage.py billing_report` · `sync_domains`	বাণিজ্যিক রিপোর্ট · DNS ছক	প্রতিদিন সকালে · ডোমেইন যোগে

প্রজেক্টে চারটি কাস্টম ম্যানেজমেন্ট কমান্ড: `seed_demo` · `seed_plans` · `billing_report` · `sync_domains` — প্রতিটির `--help` লেখা আছে, কে কখন চালাবে তা এই ছকে।

শিক্ষকের নোট: ক্লাসে `python manage.py billing_report --help` চালিয়ে দেখান — স্ব-দস্তাবেজিত কমান্ডের সুবিধা।

গঠন

`docs/` ফোল্ডার কেমন হবে

আজ কী আছে, কাল কী যোগ হবে

```
docs/
  adr/                       # নতুন
    0001-modular-monolith.md
  manual/
    user-manual-bn.html
  screenshots/     (৭৭ PNG)
  catalogue_data.py
  gen_feature_catalogue.py
  gen_proof_pdf.py
  gen_roadmap.py
  build_status.py
  deploy_check.sh
  shoot.py
  url-map.md                 # নতুন
  README.md                  # নতুন: কোন ফাইল কীসের
```

docs/README.md কেন

দশটি ফাইল থাকলে কেউ জানে না কোনটি পড়বে — একটি সূচি সব সমস্যা মেটায়।

স্ক্রিপ্ট আর দস্তাবেজ আলাদা

যা চালাতে হয় আর যা পড়তে হয় — দুই ধরনের ফাইল একই ফোল্ডারে থাকলে নামে পার্থক্য রাখুন।

যা ফেলে রাখবেন না

`old/`, `backup/`, `final_v2/` — পুরোনো দস্তাবেজ মুছুন বা আর্কাইভে সরান।

শিক্ষকের নোট: এই গঠনটি ক্লাসেই তৈরি করুন: `docs/adr/`, `docs/url-map.md`, `docs/README.md` — তিনটি ফাইল, বিশ মিনিট।

অনুশীলন ও আলোচনা

২০ মিনিট — কোডে দস্তাবেজ যোগ করুন

চারটি কাজ, সবই কমিট করার মতো

১. একটি মডিউল-ডকস্ট্রিং

নিজের সবচেয়ে জটিল ফাইলে: উদ্দেশ্য · নিয়ম · কেন এখানে।

২. একটি "কেন" মন্তব্য

যে লাইনটি অন্য কেউ "অপ্রয়োজনীয়" ভেবে মুছে দিতে পারে।

৩. CHANGELOG-এর প্রথম এন্ট্রি

আজকের কাজ, মাইগ্রেশনসহ।

৪. একটি ADR

নিজের প্রজেক্টের এক সিদ্ধান্ত, চার শিরোনামে।

- সব ফাংশনে docstring দরকার?
- ADR কেন সম্পাদনা করবেন না?
- CHANGELOG-এ মাইগ্রেশন কেন আলাদা?
- URL মানচিত্র কে ব্যবহার করবে?
- দস্তাবেজ ড্রিফট ধরবেন কীভাবে?

শিক্ষকের নোট: চারটি কাজই ছোট; ক্লাসেই করে ফেলা যায়। যেটি করা যায়নি, সেটি পরের ক্লাসের শুরুতে দেখে নিন।

সারমর্ম

ছয়টি কথা সঙ্গে নিন

ডেভেলপার-দস্তাবেজের সারাংশ

নীতি	প্রমাণ
মডিউল-ডকস্ট্রিং: উদ্দেশ্য · নিয়ম · কেন	`scoping.py` · `assessments/models.py`
মন্তব্যে "কেন", "কী" নয়	`middleware.py`-র সেশন-স্কোপ মন্তব্য
docstring যেখানে টাকা/নিরাপত্তা জড়িত	৭৫ / ৫৭৬ = ১৩% — ফাঁক চিহ্নিত
নামই প্রথম দস্তাবেজ	`test_server_refuses_the_third_student`
CHANGELOG + ADR + CONTRIBUTING	তিনটিই এখনো নেই — এই সপ্তাহের কাজ
প্রতি রিলিজে চার ধাপের লুপ	কোড → টেস্ট → দস্তাবেজ → প্রমাণ

পরের ক্লাস (৩৫): প্রমাণের দস্তাবেজ ও উপস্থাপন — স্ক্রিনশট, প্রমাণ-প্যাক, এবং পর্ব ৮-এর সমাপ্তি।

শিক্ষকের নোট: শেষে একটি কাজ দিন: "আজ একটি ADR ফাইল তৈরি করুন" — ছোট কাজ, কিন্তু অভ্যাস তৈরি হয়।

পর্ব ৮ · ক্লাস ৩৫

প্রমাণ দেখানো

প্রমাণ-প্যাক · তিনটি PDF · ডেমো স্ক্রিপ্ট · এক-পাতার সারসংক্ষেপ · রিপোজিটরি স্বাস্থ্য

IMS · ক্লাস ৩৫ · স্লাইড ৩১–৪৪

শুরু

"আমরা বানিয়েছি" — প্রমাণ কোথায়?

তিনটি PDF, ১৩৩ পাতা, ১১.৫ মেগাবাইট

Build_Proof.pdf

৩২ পাতা · ৩.৮৭ MB — কোন ফিচার, কোন টেস্ট, কোন কমান্ডের আউটপুট।

IMS_Feature_Catalogue.pdf

৭০ পাতা · ৪.৮৭ MB — ৫৭ ফিচার, প্রতিটির স্ক্রিনশট ও অবস্থা।

IMS_User_Manual_BN.pdf

৩১ পাতা · ২.৮৪ MB — বাংলায় ব্যবহারবিধি, ব্যবহারকারীর হাতে দেওয়ার মতো।

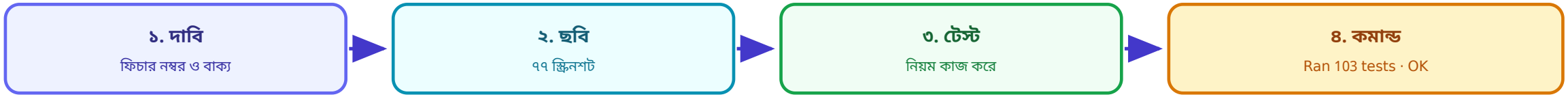
এই তিনটি কেউ হাতে লেখেনি — `docs/gen_proof_pdf.py`, `gen_feature_catalogue.py` আর `manual/print_manual.py` তৈরি করে। তাই সংখ্যা বদলালে প্রমাণও বদলায়।

শিক্ষকের নোট: ক্লাসে তিনটি PDF খুলে দেখান — বিশেষ করে স্ক্রিনশটের পাতা, যেখানে ফিচারের নামের নিচে আসল পর্দা। "এটাই প্রমাণ, কথা নয়।"

প্রমাণ-প্যাক

চার স্তরের প্রমাণ

দাবি → ছবি → টেস্ট → কমান্ড



স্তর	কী দেখায়	IMS-এ কোথায়
১. দাবি	"মার্কস প্রকাশের আগে শিক্ষার্থী দেখতে পায় না"	`catalogue_data.py` F-নম্বর
২. ছবি	পর্দাটি সত্যিই আছে	`docs/screenshots/` — ৭৭ PNG
৩. টেস্ট	নিয়মটি সত্যিই কাজ করে	`test_results_are_invisible_until_published` (৪০৩ → ২০০)
৪. কমান্ড	পুরো সিস্টেম একসাথে চলে	`Ran 103 tests … OK` · `55 pages × 4 roles`

একটি দাবি, তিনটি প্রমাণ — ছবি ছাড়া বিশ্বাস হয় না, টেস্ট ছাড়া ভরসা হয় না, কমান্ড ছাড়া পুরোটা দেখা যায় না।

যেটি নেই: পারফরম্যান্স ও লোড-টেস্টের প্রমাণ — "৮০০ ছাত্রে কেমন চলে" এখনো মাপা হয়নি।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন নিজের প্রজেক্টে এই চার স্তরের একটি ছক বানাতে — কোথায় প্রমাণ নেই, সাথে সাথে দেখা যাবে।

Build_Proof

৩২ পাতার প্রমাণ-নথি

কীভাবে সাজানো, কীভাবে পড়া

কাঠামো

ভূমিকা → ফিচার-অবস্থা সারণি → স্ক্রিনশট প্রমাণ → টেস্ট ও কমান্ড আউটপুট → সীমা ও বাকি কাজ

তৈরি হয় কীভাবে

`docs/gen_proof_pdf.py` (১৯ KB) — reportlab দিয়ে, ছবি ও সংখ্যা একসাথে

কে পড়ে

গ্রাহক, শিক্ষক, মূল্যায়নকারী — যে কেউ "সত্যিই হয়েছে?" জিজ্ঞেস করে

কেন PDF

পাঠানোর জন্য সহজ, প্রিন্ট করা যায়, রূপ বদলায় না — দস্তাবেজের "ছবি"

```
Build_Proof.pdf — ৩২ পাতা

১.  শিরোনাম + তারিখ
২.  প্রযুক্তি ও সংখ্যা
৩.  ফিচার-অবস্থা সারণি
৪–২০. ফিচার + স্ক্রিনশট
২১. টেস্ট ফলাফল
২২. smoke_test ফলাফল
২৩. check --deploy
২৪. সীমা (যা নেই)
২৫–৩২. পরিশিষ্ট
```

শিক্ষকের নোট: "কমান্ডের আউটপুট কপি-পেস্ট নয়, স্ক্রিপ্ট থেকে" — এটাই নথিটিকে বিশ্বাসযোগ্য করে। ক্লাসে `gen_proof_pdf.py` চালিয়ে দেখান।

ক্যাটালগ

৭০ পাতার ফিচার ক্যাটালগ

৫৭টি ফিচার — কোনটি তৈরি, কোনটি বাকি

অংশ	বিষয়	প্রমাণ
F-01 … F-09	অ্যাকাউন্ট ও প্রতিষ্ঠান	লগইন, রেজিস্ট্রেশন, ওয়ার্কস্পেস সুইচ
F-10 … F-15	ভূমিকা ও আমন্ত্রণ (আংশিক)	ভূমিকা আছে, ইনভাইট নেই
F-16 … F-30	একাডেমিক কাঠামো	ক্লাস, বিষয়, শিক্ষাবর্ষ, টার্ম
F-31 … F-45	উপস্থিতি, পাঠ, হোমওয়ার্ক	নেওয়া, সংশোধন, জমা, রিভিউ
F-46 … F-57	মূল্যায়ন, রিপোর্ট, বিলিং	নম্বর, প্রকাশ-গেট, CSV/XLSX/PDF, প্ল্যান

দুই ফাইলের ভূমিকা: `catalogue_data.py` (৩১ KB) = পরিকল্পনা · `build_status.py` = প্রমাণের মানচিত্র (৫৪টি ফিচারের ছবি)।

সৎ অবস্থা: ৫৭-এর ৫৪টির ছবি-প্রমাণ; বাকি তিনটি লিখিত আছে, তৈরি নয় — ক্যাটালগেই সেভাবেই আছে।

শিক্ষকের নোট: ক্যাটালগের একটি পাতা খুলে দেখান — ফিচারের বর্ণনা, অবস্থা আর স্ক্রিনশট একই পাতায়। এতে "কতটা হয়েছে" প্রশ্নের উত্তর এক নজরে।

ব্যবহারবিধি

৩১ পাতার বাংলা ম্যানুয়াল

HTML থেকে PDF — এক উৎস, দুই রূপ

```
docs/manual/
  user-manual-bn.html     # মূল (৮ অধ্যায়)
  print_manual.py         # PDF বানায়

# চালানোর নিয়ম
python docs/manual/print_manual.py
#  -> docs/IMS_User_Manual_BN.pdf  (৩১ পাতা)

# কেন HTML মূল
#  ব্রাউজারে পড়া যায়, PDF-এ ছাপা যায়,
#  ভবিষ্যতে ওয়েব-সহায়তাও একই ফাইল থেকে
```

ভূমিকা ধরে অধ্যায়

অ্যাডমিন · কোঅর্ডিনেটর · শিক্ষক · শিক্ষার্থী · প্লাটফর্ম — প্রত্যেকে নিজের অংশ পড়ে।

প্রশ্ন-উত্তর অংশ আছে

"লগইন হচ্ছে না", "মার্কস সংরক্ষণ হচ্ছে না" — সাপোর্ট কল কমানোর সবচেয়ে সস্তা উপায়।

ফাঁক

UI বদলালে HTML-এর ছবি ও লেখা হাতে বদলাতে হয় — ম্যানুয়ালের কোনো স্বয়ংক্রিয় পরীক্ষা নেই।

শিক্ষকের নোট: শিক্ষার্থীদের বলুন ম্যানুয়ালের একটি অধ্যায় পড়ে দেখতে — তারপর জিজ্ঞেস করুন: "কোথায় আটকে গেলেন?" সেই জায়গাটাই লেখার কাজ।

নিয়ম

স্ক্রিনশট-প্রমাণের পাঁচ শর্ত

ছবি সহজে মিথ্যা বলে — তাই নিয়ম

শর্ত	কারণ
আসল অ্যাপ থেকে, আসল ডেটায়	হাতে বানানো মকআপ একদিনে ধরা পড়ে
তারিখ লেখা থাকুক	"এখনো আছে?" প্রশ্নের উত্তর পাওয়া যায়
এক ছবি, এক দাবি	এক ছবিতে পাঁচ ফিচার দেখালে কিছুই প্রমাণ হয় না
ঝাপসা / কাটা নয়	পড়া না গেলে প্রমাণ নয়, সাজসজ্জা
গোপন তথ্য থাকবে না	ডেমো পাসওয়ার্ড, আসল ছাত্রের নাম-ফোন নয়

IMS-এ ডেটা: `seed_demo` নাম (Rafid Hasan, Sunrise Model School) — কাল্পনিক, তাই ছবি দেখানো নিরাপদ।

সতর্কতা: আসল প্রতিষ্ঠানের ডেটা দিয়ে ডেমো করলে আগে অনুমতি নিন, নইলে স্ক্রিনশটেই গোপনীয়তা ভাঙে।

শিক্ষকের নোট: পঞ্চম শর্তটি বাস্তবে সবচেয়ে বেশি ভাঙা হয় — ক্লাসে একটি পুরোনো স্ক্রিনশটে ডেমো পাসওয়ার্ড দেখা যায় কি না খুঁজতে বলুন।

ডেমো

১০ মিনিটের উপস্থাপনা স্ক্রিপ্ট

পাঁচ ধাপ — প্রতিটির একটি প্রমাণ

মিনিট	কী করবেন	যা প্রমাণ হয়
০–১	লগইন, চার ভূমিকার ড্যাশবোর্ড	এক অ্যাপ, চার রকম পর্দা
১–৩	শিক্ষক: উপস্থিতি নিন → সংশোধন করুন	নীতি + অডিটে জমা
৩–৬	নম্বর এন্ট্রি → প্রকাশ → শিক্ষার্থীর পর্দায়	প্রকাশ-গেট (৪০৩ → ২০০)
৬–৮	রিপোর্ট কার্ড PDF + CSV রফতানি	আসল নথি, আসল ফরম্যাট
৮–১০	অডিট লগ + সাবস্ক্রিপশন ব্যানার	জবাবদিহি + বাণিজ্যিক নিয়ম

নিয়ম: ডেমোতে **নতুন কিছু বানাবেন না** — আগেই তৈরি ডেটা ব্যবহার করুন। আর প্রতিটি ধাপে এক বাক্যে বলুন "এতে কী প্রমাণ হচ্ছে"।

শিক্ষকের নোট: ক্লাসে দুই জনকে ১০ মিনিটের ডেমো করতে দিন — সময় কমিয়ে ৫ মিনিটে নামিয়ে আনতে বলুন; এতে অপ্রয়োজনীয় অংশ বাদ পড়ে।

এক পাতা

গ্রাহকের জন্য এক-পাতার সারসংক্ষেপ

কেউ ২০ পাতার নথি পড়ে না

১. এটি কী

একটি প্রতিষ্ঠানের উপস্থিতি, নম্বর, রিপোর্ট কার্ড ও সিলেবাস — এক জায়গায়।

২. কে কী পাবে

চার ভূমিকা — অ্যাডমিন, কোঅর্ডিনেটর, শিক্ষক, শিক্ষার্থী।

৩. প্রমাণ

১০৩ টেস্ট · ৫৫ পাতা × ৪ ভূমিকা · ৭৭ স্ক্রিনশট · ৩২ পাতার প্রমাণ-নথি।

৪. দাম ও শর্ত

৳১,৫০০ / ৳৩,৫০০ / ৳৬,৫০০ — ১৪ দিনের ট্রায়াল, সিট-সীমা, মেয়াদ শেষে read-only।

৫. যা এখনো নেই

ইনভাইট · CSV আমদানি · অনলাইন পেমেন্ট — সৎভাবে লেখা।

কেন ৫ নম্বর থাকবে: যা নেই তা আগেই বলা থাকলে পরে অভিযোগ আসে না — আর ভরসা বাড়ে।

দৈর্ঘ্য: এক A4 পাতা; তার বেশি হলে কেউ শেষ পর্যন্ত পড়ে না।

শিক্ষকের নোট: শিক্ষার্থীদের এক-পাতার সারসংক্ষেপ লিখিয়ে নিন — সবচেয়ে কঠিন কাজ, কারণ সব বাদ দিয়ে মূল কথাটি রাখতে হয়।

রিপোজিটরি

প্রথম নজরে যা দেখা যায়

কেউ রিপো খুললে ৩০ সেকেন্ডে কী বোঝে

ফাইল	আছে?	কেন দরকার
`README.md`	আছে (বাসি)	প্রথম পরিচয় — হালনাগাদ করা জরুরি
`START_HERE.md`	আছে	চালানোর পথ, দুই বিকল্প
`.gitignore`	আছে	`.env`, `media/`, `backups/`, `__pycache__` বাদ
`.env.example` · `.env.production.example`	আছে	কী কী মান লাগবে, গোপন মান ছাড়া
`Procfile` · `build.sh`	আছে	হোস্ট কীভাবে চালাবে
`docs/`	আছে	প্রমাণ, ম্যানুয়াল, স্ক্রিনশট, জেনারেটর
`CHANGELOG.md` · `CONTRIBUTING.md` · `LICENSE`	**নেই**	সংস্করণ-ইতিহাস, দলের নিয়ম, ব্যবহারের শর্ত

প্রথম চারটি সারি ঠিক থাকলে রিপো "পেশাদার" দেখায়; শেষ সারি বাকি থাকলে দীর্ঘমেয়াদে সমস্যা — বিশেষ করে লাইসেন্স, কারণ ব্যবহারের শর্ত ছাড়া কোড share করা ঝুঁকি।

শিক্ষকের নোট: লাইসেন্স নিয়ে সতর্ক করুন: প্রতিষ্ঠানের কোড কখনো ওপেন রিপোতে দেবেন না; ব্যক্তিগত প্রজেক্টে MIT/Apache-2.0 সাধারণ পছন্দ।

যাচাই

কেউ প্রশ্ন করলে কোন প্রমাণ দেখাবেন

ছয়টি প্রশ্ন, ছয়টি উত্তর — তৈরি রাখুন

প্রশ্ন	প্রমাণ
"কোড কাজ করে?"	`Ran 103 tests in 51.469s · OK`
"পাতাগুলো ভাঙে না?"	`smoke_test.py` — ৫৫ পাতা × ৪ ভূমিকা
"সার্ভারে চলবে?"	`check --deploy` → no issues · `deploy_check.sh`
"ডেটা নিরাপদ?"	স্কোপিং টেস্ট · ৪০৪ ক্রস-ওয়ার্কস্পেস · `AuditLog`
"ভুল হলে?"	রোলব্যাক + `ops/backup.sh` + restore ড্রিল
"আমাদের স্কুলে চলবে?"	২০ মিনিটের অনবোর্ডিং (LAUNCH_PLAYBOOK §4) + রিপোর্ট কার্ড

প্রস্তুতি: এই ছয়টি কমান্ডের আউটপুট এক ফোল্ডারে রেখে দিন (`docs/proof/`) — প্রশ্নের মুহূর্তে খুঁজতে হবে না।

শিক্ষকের নোট: ক্লাসে প্রতিটি কমান্ড একবার চালিয়ে আউটপুট সংরক্ষণ করুন — এটাই "প্রমাণ-ফোল্ডার" বানানোর সবচেয়ে সহজ উপায়।

পর্ব ৮ সমাপ্তি

তিন ক্লাসে যা শেখা হলো

৪৪ স্লাইড · ছয়টি দস্তাবেজ · তিনটি প্রমাণ-নথি

ক্লাস	মূল ধারণা	প্রমাণ
৩৩ · কে পড়বে	চার পাঠক · ব্যবহারবিধি · স্ক্রিনশট	৯০০ লাইন · ৭৭ PNG · ৫৭ ফিচার
৩৪ · কোডের ভেতরে	docstring · "কেন" মন্তব্য · CHANGELOG · ADR	৭৫/৫৭৬ docstring · ৮৫ রুট
৩৫ · প্রমাণ দেখানো	চার স্তরের প্রমাণ · ডেমো · এক-পাতা	৩২+৭০+৩১ পাতা · ১১.৫ MB

সৎ ফাঁক: ADR ফোল্ডার · CHANGELOG · CONTRIBUTING · LICENSE · `docs/url-map.md` · `rebuild_docs.sh` · লোড-টেস্টের প্রমাণ

পর্ব ৯-এ: ভবিষ্যৎ — কোন ফিচার আগে, কীভাবে অনুমান করবেন, আর কোথায় থামবেন।

শিক্ষকের নোট: ফাঁকের তালিকা থেকে প্রত্যেককে দুটি বেছে নিয়ে পরের সপ্তাহে করতে বলুন — ছোট, তবু শেষ হয়।

অনুশীলন ও আলোচনা

২০ মিনিট — প্রমাণ-প্যাক বানান

নিজের প্রজেক্টের জন্য চারটি জিনিস

১. প্রমাণ ফোল্ডার

চারটি কমান্ডের আউটপুট → `docs/proof/`

২. দাবি-ছবি-টেস্ট ছক

পাঁচটি ফিচার → ছবি → টেস্টের নাম

৩. ৫ মিনিটের ডেমো

লিখিত স্ক্রিপ্ট, প্রতিটি ধাপে "কী প্রমাণ হচ্ছে"

৪. এক-পাতার সারসংক্ষেপ

পাঁচটি শিরোনাম, এক A4

- দাবির সঙ্গে কয়টি প্রমাণ?
- স্ক্রিনশটে কী থাকবে না?
- এক-পাতায় কী বাদ দেবেন?
- "চলবে?" প্রশ্নের প্রথম উত্তর?
- লাইসেন্স না থাকলে কী ঝুঁকি?

শিক্ষকের নোট: চতুর্থ প্রশ্নের উত্তর: টেস্টের সংখ্যা ও `smoke_test` — তারপর অনবোর্ডিং। "কথা নয়, আউটপুট।"

৩৬

পর্ব ৯ — ভবিষ্যৎ

কোন ফিচার আগে · অনুমান বনাম প্রমাণ · ব্যবহারকারীর কাছ থেকে শেখা · কখন থামবেন